

```

    if (j > 0) {
        tokens[token_count][j] = '\0';
        token_count++;
    }

    printf("\n--- Parsed Arguments ---\n");

    for (i = 0; i < token_count; i++) {
        printf("Argument %d: %s\n", i + 1, tokens[i]);
    }

    return 0;
}

```

```

nano task1.c teja@LAPTOP-FGDPB110:~$ nano task1.c
teja@LAPTOP-FGDPB110:~$ gcc task1.c -o task1
teja@LAPTOP-FGDPB110:~$ ./task1
myshell> myshell> echo Hello

--- Parsed Arguments ---
Argument 1: myshell>
Argument 2: echo
Argument 3: Hello
teja@LAPTOP-FGDPB110:~$ |

```

Theory

Tokenization is the process of breaking the command entered by the user into smaller units called **tokens**.

TASK 2:

```
GNU nano 7.2 task2.c *
#include <stdio.h>
#include <string.h>

int main() {
    char input[200];
    char result[200];

    printf("myshell> ");
    fgets(input, sizeof(input), stdin);

    int i = 0, j = 0;
    int in_single_quote = 0;

    while (input[i] != '\0' && input[i] != '\n') {
        if (input[i] == '\'' ) {
            in_single_quote = !in_single_quote;
        } else {
            result[j++] = input[i];
        }

        i++;
    }

    if (in_single_quote) {
        printf("\nSyntax Error: Unmatched single quote.\n");
        return 1;
    }

    result[j] = '\0';

    printf("\nParsing Result:\n");
    printf("%s\n", result);

    return 0;
}
```

```
teja@LAPTOP-FGDPB110:~$ nano task2.c
teja@LAPTOP-FGDPB110:~$ gcc task2.c -o task2
teja@LAPTOP-FGDPB110:~$ ./task2
myshell> myshell> echo 'Hello World'

Parsing Result:
myshell> echo Hello World
teja@LAPTOP-FGDPB110:~$ myshell> echo 'Hello World'
```

Theory

Single quotes are used to preserve the **literal content** inside them.

Task 3:

```
GNU nano 7.2 task3.c
#include <stdio.h>

int main()
{
    char input[100];
    int length = 0;
    char ch;

    printf("Enter command: ");

    while (1)
    {
        ch = getchar();

        if (ch == '\n')
        {
            input[length] = '\0';
            break;
        }
        else if (ch == 8 || ch == 127)
        {
            if (length > 0)
            {
                length--;
                printf("\b \b");
            }
        }
        else
        {
            if (length < 99)
            {
                input[length] = ch;
                length++;
                putchar(ch);
            }
        }
    }
}
```

Theory

Double quotes allow spaces to remain part of the same argument while still allowing **variable expansion**

```
teja@LAPTOP-FGDPB110:~$ nano task3.c
teja@LAPTOP-FGDPB110:~$ gcc task3.c -o task3
teja@LAPTOP-FGDPB110:~$ ./task3
Enter command: hello world
hello world
You entered: hello world
teja@LAPTOP-FGDPB110:~$ myshell> echo "Hello World"
```

Theory

Double quotes allow spaces to remain part of the same argument while still allowing **variable expansion**

Task 4

```
#include <stdio.h>
#include <string.h>

int main() {
    char input[200];
    char result[200];

    printf("myshell> ");
    fgets(input, sizeof(input), stdin);

    int i = 0, j = 0;
    int in_single_quote = 0;

    while (input[i] != '\0' && input[i] != '\n') {
        if (input[i] == '\\') {
            in_single_quote = !in_single_quote;
        } else {
            result[j++] = input[i];
        }

        i++;
    }

    if (in_single_quote) {
        printf("\nSyntax Error: Unmatched single quote.\n");
        return 1;
    }

    result[j] = '\0';

    printf("\nParsing Result:\n");
    printf("%s\n", result);

    return 0;
}
```



```
teja@LAPTOP-FGDPB110:~$ nano task5.c
teja@LAPTOP-FGDPB110:~$ gcc task5.c -o task5
teja@LAPTOP-FGDPB110:~$ ./task5
myshell$ myshell$ ls
Parent Process PID: 459
Created Child PID: 460
Child Process PID: 460
Execution failed: No such file or directory
Child exited with status: 2
teja@LAPTOP-FGDPB110:~$ myshell$ ls @l
```

Task 5 is about creating and managing child processes. A parent process creates a child process using `fork()`. The child process executes the required program using `execvp()`, while the parent process waits for the child to finish using `waitpid()`.

The program also passes arguments to the child process and handles execution errors if the command cannot be run.

In simple words: **Parent creates child → Child runs the command → Parent waits → Child finishes → Parent continues.**